

Big Data : Flashcard complet

? 01-Q : C'est quoi le Big Data ?

R : Le **Big Data** désigne un volume très important de données, souvent trop grand ou complexe pour être traité par les outils traditionnels. Il se caractérise par trois aspects principaux appelés les **3V** :

- **Volume** : des quantités massives de données, allant jusqu'aux téraoctets et pétaoctets.
- **Vélocité** : la vitesse à laquelle ces données sont générées et doivent être traitées, souvent en quasi temps réel.
- **Variété** : la diversité des types de données, qui peuvent être structurées (bases de données classiques), semi-structurées (logs, fichiers XML) ou non structurées (vidéos, images, textes libres).

Le Big Data nécessite des infrastructures comme Hadoop pour stocker ces données sur plusieurs machines et des frameworks comme MapReduce pour les analyser efficacement.

? 02-Q : Quels sont les enjeux du BD ?

R :

Les enjeux du Big Data sont principalement :

1. **Gestion du volume** : Traiter, stocker et analyser d'immenses quantités de données (tétra-, pétaoctets et plus) efficacement.
2. **Traitement rapide (vélocité)** : Analyser les données qui arrivent à très grande vitesse, souvent en temps quasi réel, pour prendre des décisions immédiates.
3. **Gestion de la variété** : Intégrer et exploiter des données de formats très différents (texte, vidéo, audio, fichiers, bases relationnelles ou non).
4. **Scalabilité** : Pouvoir étendre les systèmes de stockage et de calcul sur plusieurs machines sans reprogrammer les algorithmes.
5. **Tolérance aux pannes** : Garantir la fiabilité du système malgré la défaillance possible de certains serveurs, grâce à des mécanismes comme ceux d'Hadoop.

6. **Exploitation des données non structurées** : Transformer des données brutes et disparates en informations exploitables.
7. **Coût** : Mettre en place une infrastructure économique pour gérer cette masse de données sans exploser le budget.

Ces enjeux poussent à utiliser des technologies adaptées (Hadoop, MapReduce, YARN, NoSQL) pour répondre aux besoins spécifiques du Big Data.

? 03-Q : Quels sont les domaines d'application du Big Data ?

R :

Le Big Data est utilisé dans de nombreux secteurs pour extraire de la valeur des énormes volumes de données. Parmi les principaux domaines :

1 Analyse du comportement des clients

- Analyse des **journaux web** des sites e-commerce pour comprendre la navigation et optimiser les ventes.
- Analyse des **journaux d'appel** des abonnés dans les télécoms pour améliorer les services.
- Analyse des **journaux d'achat** dans les supermarchés pour mieux gérer les stocks et les promotions.

2 Systèmes de recommandation

- Pour suggérer des produits ou services aux clients en fonction de leurs comportements et préférences (comme sur Amazon ou Netflix).

3 Secteur de la vente au détail (Retail)

- Pour prévoir quels articles seront les plus demandés, améliorer l'expérience client et optimiser la logistique.

4 Ciblage publicitaire

- Sur les réseaux sociaux, les données des profils et des activités sont analysées pour proposer des publicités adaptées aux goûts et comportements des utilisateurs.

? 04-Q : Quels sont les défis du Big Data ?

R :

Travailler avec de très grandes quantités de données entraîne plusieurs défis :

◆ 1. Capture des données

- Collecter efficacement des données provenant de multiples sources variées et en continu (IoT, réseaux sociaux, logs, etc.).

◆ 2. Stockage des données

- Trouver des solutions fiables et économiques pour stocker des volumes massifs de données tout en garantissant leur disponibilité.

◆ 3. Traitement des données

- Traiter les données rapidement (parfois en temps réel) malgré leur volume, leur variété et leur vitesse d'arrivée.

◆ 4. Analyse des données

- Extraire des informations pertinentes à partir de données complexes et non-structurées, et produire des résultats compréhensibles et exploitables.

◆ 5. Confidentialité et sécurité

- Protéger les données sensibles des utilisateurs et garantir la confidentialité et la conformité aux réglementations (comme le RGPD en Europe).

? 05-Q : C'est quoi le BD Analytics (BDA) ?

R :

Le **Big Data Analytics** désigne l'ensemble des **techniques et outils** utilisés pour **analyser et extraire des informations utiles** à partir de très grands volumes de données (Big Data).

Cela permet de **découvrir des tendances, des modèles, des corrélations**, voire de faire des **prédictions** pour aider à la prise de décision.

En résumé, c'est le traitement et l'analyse des données massives pour en tirer de la valeur.

? 06-Q : Pourquoi le BDA est elle importante ?

R :

L'analyse du Big Data (BDA) est importante parce qu'elle permet de **tirer de la valeur** de la masse énorme de données disponibles aujourd'hui.

Concrètement, elle est utile pour :

- ✓ Découvrir des **tendances et des modèles** cachés dans les données.
- ✓ Prendre de **meilleures décisions**, basées sur des faits et non sur des intuitions.
- ✓ Anticiper des événements grâce à la **prédiction** (exemple : panne de machine, comportement des clients).
- ✓ Améliorer la **performance** et l'**efficacité** des entreprises (optimisation des coûts, des opérations).
- ✓ Offrir des services ou produits plus personnalisés aux clients.

Donc, sans analyse, le Big Data n'est qu'un amas de données inutilisables. C'est l'analyse qui transforme ces données en **connaissance et avantage compétitif**.

? 07-Q : Donne des exemples de domaines où l'on peut utiliser le BDA

R :

Banque et finance

- Détection des fraudes en temps réel.
- Analyse du risque de crédit.
- Personnalisation des offres pour les clients.

Commerce / grande distribution

- Étudier les comportements d'achat des clients.
- Optimiser les stocks et la chaîne logistique.
- Proposer des recommandations personnalisées.

Télécommunications

- Analyser l'utilisation du réseau pour anticiper les surcharges.
- Fidéliser les clients en comprenant pourquoi ils partent.

Santé

- Prédire des épidémies ou des maladies.
- Aider au diagnostic médical grâce à l'analyse des dossiers et imageries.

Transport / logistique

- Optimiser les itinéraires et réduire les coûts de carburant.
- Suivre les véhicules et les livraisons en temps réel.

Médias sociaux et web

- Comprendre les tendances et les opinions des utilisateurs.
- Analyser les sentiments dans les publications.

Industrie

- Maintenance prédictive des machines.
- Optimisation des processus de production.

? 08-Q : Cite et décris les 8 phases du cycle de vie du BDA

R :

1 Évaluation de l'analyse de rentabilisation

C'est le point de départ. On définit la raison pour laquelle on fait l'analyse et ce qu'on veut atteindre (objectif, bénéfices attendus).

➡ *Exemple : Améliorer la fidélité des clients, réduire les coûts, etc.*

2 Identification des données

On cherche et liste toutes les sources de données pertinentes à analyser.

➡ *Exemple : logs web, CRM, réseaux sociaux, capteurs IoT...*

3 Filtrage des données

On nettoie les données pour enlever celles qui sont corrompues, incomplètes, inutiles ou incohérentes.

➡ *Exemple : supprimer les doublons, corriger des valeurs aberrantes.*

4 Extraction des données

Les données brutes sont transformées dans un format compatible avec les outils d'analyse utilisés.

➡ *Exemple : convertir des fichiers texte en tables, changer des formats, extraire les colonnes utiles.*

5 Agrégation des données

On regroupe et intègre les données venant de différentes sources, en alignant les champs identiques pour les rendre cohérentes et exploitables ensemble.

➡ *Exemple : associer les ventes en ligne et en magasin par client.*

6 Analyse des données

On applique des méthodes statistiques ou analytiques pour trouver des tendances, des corrélations, des modèles.

➡ *Exemple : analyser les comportements d'achat, détecter des fraudes.*

7 Visualisation des données

Les résultats sont présentés sous forme de graphiques, tableaux de bord, cartes, pour être plus facilement compris.

➡ *Outils comme Tableau, Power BI, QlikView.*

8 Résultat final de l'analyse

Les conclusions sont communiquées aux décideurs (parties prenantes), qui vont utiliser ces résultats pour prendre des mesures concrètes.

? 09-Q : Cite et décris les 4 types de BDA

R :

1 Analyse descriptive

Elle résume les **données passées** sous une forme facile à lire.

Elle sert à produire des rapports (revenus, bénéfices, ventes, etc.) ou des statistiques sur les réseaux sociaux.

➡ *Exemple : Dow Chemical a identifié de l'espace sous-utilisé dans ses bureaux grâce à cette analyse, économisant 4M\$/an.*

2 Analyse diagnostique

Elle cherche à comprendre les **causes** d'un problème.

On utilise des techniques comme l'exploration de données ou l'analyse approfondie.

➡ *Exemple : Une baisse des ventes d'un site e-commerce malgré des paniers remplis peut être diagnostiquée en identifiant des frais de port élevés, des bugs du formulaire, etc.*

3 Analyse prédictive

Elle utilise les données passées et actuelles pour **prédire l'avenir**.

Elle s'appuie sur l'IA, le machine learning, et des algorithmes pour prévoir des tendances (clients, marchés, comportements).

➡ *Exemple : PayPal prévoit les fraudes en analysant les comportements et données historiques.*

4 Analyse prescriptive

Elle propose la **meilleure solution possible** à un problème donné.

Elle combine souvent les analyses descriptive et prédictive, et utilise aussi l'IA et le machine learning.

➡ *Exemple : Une compagnie aérienne ajuste automatiquement ses tarifs en fonction de la demande, météo, saisons, etc.*

? 10-Q : Quels sont les technologies et outils clés du BDA ?

R :

✓ 1. Hadoop et son écosystème

- Framework open source pour stocker et traiter de grands ensembles de données.
 - Gère des données **structurées** et **non structurées**.
 - Écosystème : inclut HDFS, MapReduce, Hive, Pig, etc.
 - Permet de distribuer le stockage et le calcul sur plusieurs machines.
-

✓ 2. MapReduce

- Modèle de programmation (souvent associé à Hadoop) qui divise le traitement en deux étapes :
 - ◆ *Map* (partition des données et calcul local)
 - ◆ *Reduce* (agrégation des résultats).
 - Idéal pour le traitement parallèle.
-

✓ 3. Bases de données NoSQL

- Bases de données non relationnelles, sans schéma fixe, adaptées aux données massives et non structurées.
 - Exemples : MongoDB, Cassandra, Couchbase, etc.
-

✓ 4. Analyse prédictive

- Outils logiciels qui exploitent le **machine learning** et des algorithmes statistiques pour prévoir des événements futurs (fraude, risque, comportement client).
-

✓ 5. Outils d'analyse de données en flux (stream data)

- Pour analyser des données en temps réel.
 - Ils filtrent, agrègent et analysent les données dès leur arrivée.
 - Exemples : Apache Storm, Apache Kafka.
-

✓ 6. Stockage distribué

- Les données sont répliquées sur plusieurs nœuds pour assurer tolérance aux pannes et accès rapide.
 - Utilise souvent des bases NoSQL ou HDFS.
-

✓ 7. Data Lake

- Grand réservoir de stockage contenant des données brutes (structurées ou non) dans leur format natif jusqu'à leur utilisation.
-

✓ 8. Data Warehouse (entrepôt de données)

- Référentiel qui stocke des données organisées avec un schéma prédéfini, généralement pour des analyses classiques (OLAP).
-

✓ 9. Outils de data mining (fouille de données)

- Pour découvrir des motifs cachés et des informations utiles dans de grands volumes de données.

✓ 10. Virtualisation des données

- Accès aux données de différentes sources sans avoir à les déplacer ou à les répliquer.

✓ 11. Logiciel d'intégration de données

- Permet de faire circuler et transformer les données entre plateformes diverses (Hadoop, MongoDB, AWS...).

✓ 12. Logiciel de qualité et de prétraitement des données

- Nettoie, enrichit et formate les données pour analyse (exemple : suppression des doublons, correction des erreurs).

✓ 13. Apache Spark

- Framework open source, plus rapide que MapReduce, pour traiter des données **par lots** et **en streaming**.

? 11-Q : Quels sont les avantages du BDA ?

R :

1 Traitement rapide de grands volumes de données

- ➡ Capacité à analyser rapidement d'énormes quantités de données provenant de multiples sources, dans des formats variés (structurés, semi-structurés, non structurés).

2 Prise de décisions plus rapides et mieux informées

- ➡ Grâce aux informations extraites, les entreprises peuvent élaborer des stratégies efficaces, optimiser leurs opérations et réagir plus vite aux changements.

3 Économies de coûts

- ➡ L'optimisation des processus et l'identification d'inefficacités permettent de réduire les dépenses et d'améliorer la rentabilité.

4 Meilleure compréhension des clients

- ➡ En analysant le comportement, les besoins et les sentiments des clients, les entreprises peuvent mieux cibler leur marketing,

personnaliser leurs offres et fidéliser leur clientèle.

5 Développement et amélioration des produits

➡ Les insights tirés des données guident le développement et l'amélioration de produits qui répondent mieux aux attentes du marché.

6 Stratégies de gestion des risques plus efficaces

➡ En identifiant des schémas de risques dans les données, les entreprises peuvent anticiper et gérer les menaces plus efficacement.

? 12-Q : Quels sont les défis du BDA ?

R :

1 Accessibilité des données

➡ Plus il y a de données, plus leur stockage et leur traitement deviennent complexes.

➡ Il faut les organiser et les entretenir correctement pour que même des utilisateurs moins expérimentés puissent les exploiter.

2 Maintien de la qualité des données

➡ Les données viennent de nombreuses sources, dans des formats variés, et en grande quantité.

➡ Il est donc coûteux et long de nettoyer, valider et maintenir une qualité élevée des données.

3 Sécurité des données

➡ Les écosystèmes Big Data sont complexes, et assurer la confidentialité, l'intégrité et la sécurité dans un tel environnement est difficile.

4 Choix des outils appropriés

➡ Avec la multitude d'outils et plateformes disponibles, il est parfois difficile de sélectionner ceux qui conviennent le mieux aux besoins et à l'infrastructure de l'entreprise.

5 Manque de compétences et coût des experts

➡ Le manque de data scientists et d'ingénieurs qualifiés ainsi que le coût élevé pour les recruter ou les former constituent un obstacle pour beaucoup d'organisations.

? 13-Q : Cite et décris les 5V du BD

R :



Figure 1: Les 5V du Big Data

1 Volume

- C'est la **quantité énorme de données** générées par des machines, réseaux, capteurs, interactions humaines, réseaux sociaux, etc.
- Ces données sont tellement massives qu'elles ne peuvent pas être stockées ni traitées sur une seule machine avec des outils classiques.
- On parle de données de l'ordre des téraoctets (To), pétaoctets (Po), exaoctets (Eo) et zettaoctets (Zo).
- Exemple : réseaux sociaux avec des milliards de messages/jour, capteurs industriels générant des To quotidiens.

2 Variété

- C'est la **diversité des formats et sources** des données.
- Les données peuvent être **structurées** (tableaux, bases SQL), **non structurées** (texte libre, vidéos, images, PDF, audio) ou **semi-structurées** (JSON, XML).
- Elle provient d'e-mails, réseaux sociaux, vidéosurveillance, capteurs, etc.

- Cette variété pose des défis de stockage, d'intégration et d'analyse.
-

3 Vitesse (ou rapidité)

- C'est la **rapidité avec laquelle les données sont générées, transmises et analysées**.
 - Les flux de données arrivent parfois en temps réel ou en millisecondes (par ex. capteurs de voiture autonome, bourse, détection de fraude).
 - Nécessite des outils capables d'analyser les données **en continu et rapidement**, sans attendre.
-

4 Véracité

- C'est la **fiabilité et précision des données**.
 - Les données doivent être nettoyées et filtrées pour enlever les erreurs, incohérences et "bruit" inutile.
 - Car des données incorrectes entraînent des analyses fausses.
 - Question centrale : « Ces données sont-elles pertinentes et exactes pour le problème analysé ? »
-

5 Valeur

- C'est la **valeur utile que l'on tire des données**.
 - Le but final est d'exploiter les données pour en dégager des informations précieuses pour l'entreprise (décisions, stratégie, efficacité, satisfaction client...).
 - Une donnée n'a de sens que si elle sert à atteindre un objectif clair.
-

? 14-Q : Cite et décris les différents type de données, avec des exemples

R :

1 Données structurées

- **◆ Définition :**
 - Ce sont des données organisées dans un format **fixe**, sous forme de lignes et colonnes, faciles à stocker et à interroger.

- Elles ont des **clés relationnelles** et des **champs prédéfinis**.
- Très adaptées aux bases de données relationnelles (SQL).
-  **Exemples :**
 - Tables SQL
 - Feuilles Excel
 - Systèmes OLTP (Online Transaction Processing)

Ces données sont les plus simples à gérer mais ne représentent que 5–10% des données mondiales.

2 Données semi-structurées

-  **Définition :**
 - Données qui n'ont pas un schéma strict comme les données structurées mais gardent une certaine **organisation** pour faciliter leur traitement.
 - Ne sont pas stockées dans des bases relationnelles classiques.
-  **Exemples :**
 - Fichiers **CSV**
 - Documents **XML**
 - Documents **JSON**

Comme les données structurées, elles représentent environ 5–10% des données mondiales.

3 Données non structurées

-  **Définition :**
 - Données qui n'ont ni schéma, ni organisation prédéfinie. Ce sont les plus courantes dans le monde (~80% des données).
 - Souvent sous forme de texte libre, images, vidéos, etc.
-  **Exemples généraux :**

- Emails
- Documents Word
- Photos, vidéos
- Fichiers audio
- Pages Web
- Présentations PowerPoint
- Messages et publications sur les réseaux sociaux
- Logs

? 15-Q : Donne des exemples spécifique de données non structurées

R :

- ✓ Images satellites (données météorologiques, surveillance, Google Earth)
- ✓ Données scientifiques (sismiques, atmosphériques, physique des particules)
- ✓ Vidéos de sécurité, trafic, surveillance
- ✓ Données radar ou sonar (profils sismiques, océanographiques)
- ✓ Documents d'entreprise (rapports, emails, journaux internes)
- ✓ Données des réseaux sociaux (Facebook, Twitter, LinkedIn, YouTube)
- ✓ Données mobiles (SMS, géolocalisation)
- ✓ Contenu Web (YouTube, Wikipedia, Vimeo...)

? 16-Q : Donne la répartition en pourcentage des différents types de données

R :

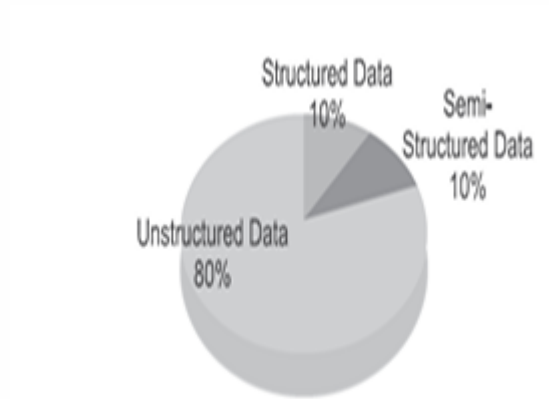


Figure : Répartition en pourcentage des différents types de données

Répartition :

- **Données non structurées : ~80 %**

La majorité des données générées aujourd'hui (textes libres, images, vidéos, réseaux sociaux, etc.).

- **Données semi-structurées : ~10 %**

Comme les fichiers XML, JSON, CSV.

- **Données structurées : ~10 %**

Comme les données en bases SQL, Excel, OLTP.

Résumé :

Type de données	Pourcentage approximatif
Non structurées	 80 %
Semi-structurées	 10 %
Structurées	 10 %

17-Q : Parle moi des technologies Big Data

R :

1. Deux grandes catégories de technologies Big Data :

- ✓ Technologies de stockage
 - ✓ Technologies de traitement et d'analyse
-

Technologies de stockage Big Data

Elles servent à **stocker d'énormes volumes de données** de manière fiable et distribuée.

➡ Principales solutions :

1 HDFS (Hadoop Distributed File System)

- Système de fichiers distribué utilisé par Hadoop.
- Permet de stocker des données massives sur plusieurs machines à faible coût.
- Tolérant aux pannes et adapté aux données structurées ou non structurées.

2 Bases de données NoSQL

- Systèmes de gestion de données non relationnelles (ex. MongoDB, Cassandra).
 - Idéal pour des données volumineuses, hétérogènes et sans schéma fixe.
 - Stocke des données structurées, semi-structurées ou non structurées.
-

Technologies de traitement et d'analyse Big Data

Elles servent à **analyser et traiter ces données à grande échelle**, en temps réel ou par lots.

➡ Principales plateformes :

1 Apache Hadoop

- Framework open source pour le stockage et traitement distribués.
- Utilise le modèle **MapReduce** pour traiter de grands ensembles de données.

2 Apache Spark

- Moteur d'analyse unifié, très rapide, qui fonctionne en mémoire (in-memory).
- Plus rapide que MapReduce pour des traitements complexes.
- Supporte le **machine learning** et le traitement de flux (streaming).
- Fonctionne avec Hadoop, Mesos ou en mode autonome.

3 Apache Storm

- Conçu pour le **traitement en temps réel** des données en streaming.
- Gère les flux continus comme les tweets, logs ou capteurs.

4 Apache Flink

- Framework pour **traitement en temps réel et batch (par lots)**.
- Flexible et puissant pour les analyses continues et les calculs complexes.

◆ Résumé rapide :

Catégorie	Technologie	Rôle
 Stockage	HDFS	Stockage distribué et fiable
 Stockage	NoSQL	Bases adaptées aux gros volumes et données variées
 Traitement	Hadoop	Traitement distribué (batch)
 Traitement	Spark	Traitement rapide en mémoire, batch & streaming
 Traitement	Storm	Streaming en temps réel
 Traitement	Flink	Streaming + batch

? 18-Q : C'est quoi un système de fichiers ?

R :

C'est un **logiciel qui organise, stocke et récupère les données sur un disque ou autre périphérique**.

Il définit comment les données sont structurées sur le support (fichiers, dossiers) et comment y accéder.

Exemples : NTFS, ext4 (basés fichiers), S3 (basé objets), cinder (basé blocs).

? 19-Q : C'est quoi un système distribué ?

R :

C'est un **groupe d'ordinateurs en réseau qui travaillent ensemble comme une seule machine** pour accomplir une tâche.

Pour l'utilisateur, le système apparaît comme un seul ordinateur, même s'il repose sur plusieurs.

Exemple : un cluster avec 4 machines de 2 cœurs chacune → apparaît comme une seule machine de 8 cœurs.

? 20-Q : C'est quoi un calcul distribué (distributed computing) ?

R :

C'est le fait de **répartir des calculs complexes sur plusieurs machines** pour les exécuter plus vite.

On utilise la puissance combinée de plusieurs ordinateurs pour résoudre un problème.

Exemple : traitement de gros volumes de données ou simulation scientifique.

? 21-Q : C'est quoi un stockage distribué ?

R :

C'est lorsque les données sont **réparties sur plusieurs disques ou machines**, mais apparaissent pour l'utilisateur comme un seul disque.

Cela augmente la capacité, la redondance et la tolérance aux pannes.

? 22-Q : C'est quoi un cluster de calcul ?

R :

C'est un **groupe d'ordinateurs (appelés nœuds)** interconnectés qui travaillent ensemble pour exécuter des tâches exigeantes.

Les clusters sont utilisés dans la recherche, l'analyse de données, le machine learning, etc.

→ Plus puissant et plus fiable qu'une seule machine.

? 23-Q : C'est quoi un espace de nom ?

R : Un espace de noms est **l'organisation des noms (ou chemins) qui permettent d'identifier et d'accéder à une donnée** dans un système (local ou distribué), comme si tout était dans un seul "espace" cohérent.

? 24-Q : C'est quoi un système de fichiers distribué (DFS) ?

R :

C'est un système de fichiers qui fournit un **espace de noms global unifié sur plusieurs machines**.

L'utilisateur accède aux fichiers sans se soucier de leur emplacement physique.

Exemple : si un fichier est sur node10, tu peux le lire depuis node5 comme s'il était local.

? 25-Q : C'est quoi l'informatique distribuée ?

R :

C'est la **gestion et l'exécution des processus d'une application sur plusieurs machines ou processeurs**, pour partager la charge et améliorer la performance.

En résumé : elle organise les calculs pour qu'ils soient réalisés simultanément sur plusieurs nœuds.

? 26-Q : C'est quoi Hadoop ?

R :

Hadoop est un **framework open-source** qui permet de **stocker et traiter de très grandes quantités de données** sur un groupe de serveurs (appelé cluster).

Il décompose les données en morceaux, les distribue sur plusieurs machines pour les stocker (dans HDFS) et les traite en parallèle (avec MapReduce ou d'autres moteurs).

Son but est de rendre possible le traitement de « Big Data » de façon efficace, même en cas de panne d'une machine.

? 27-Q : C'est quoi MapReduce ?

R :

MapReduce est un **modèle de programmation** utilisé par Hadoop pour traiter de grandes quantités de données de façon parallèle et distribuée.

Il se compose de deux phases principales :

- **Map** : chaque machine lit une partie des données, les transforme en paires clé/valeur, et les trie localement.
- **Reduce** : les paires sont regroupées par clé et traitées pour produire le résultat final.

Exemple simple :

Tu veux compter combien de fois chaque mot apparaît dans un gros livre.

- **Map** : chaque machine compte les mots dans ses pages.
- **Reduce** : on regroupe les comptages de tous les machines pour chaque mot et on additionne.

C'est efficace car chaque machine travaille en parallèle sur une partie des données.

? 28-Q : C'est quoi un cluster Hadoop ?

R :

Un **cluster Hadoop** est un groupe d'ordinateurs (appelés **nœuds**) qui travaillent ensemble pour stocker et traiter de grandes quantités de données.

Dans ce cluster :

- Certains nœuds (le **master**) gèrent et coordonnent le travail.
- Les autres (les **workers**) stockent les données et exécutent les tâches (MapReduce ou autres).

Chaque nœud utilise son disque et son processeur pour une partie du travail, et tout est coordonné pour donner un résultat global.

💡 En résumé : c'est une « ferme » de machines configurées pour collaborer et exécuter Hadoop.

? 29-Q : Comment fonctionne un cluster Hadoop ?

R :

Un cluster Hadoop fonctionne en deux grandes étapes :

1. Stocker les données

- Les fichiers sont divisés en petits blocs (par exemple 128 Mo chacun).
 - Ces blocs sont **répliqués** (copiés sur plusieurs machines pour la tolérance aux pannes).
 - Tout cela est géré par le système de fichiers distribué de Hadoop : **HDFS**.
 - Un nœud maître (NameNode) garde la carte de où se trouve chaque bloc.
 - Les nœuds workers (DataNodes) stockent réellement les blocs.
-

2. Traiter les données

- Les calculs (par exemple avec MapReduce) sont envoyés **là où les données sont stockées**, pour éviter de déplacer des gros fichiers sur le réseau.
 - Un nœud maître (JobTracker ou ResourceManager/YARN) décide qui fait quoi.
 - Les nœuds workers (TaskTrackers/NodeManagers) exécutent les tâches sur leurs données locales.
-

 En résumé :

- ✓ Tu déposes un gros fichier → Hadoop le découpe, le distribue et le réplique sur plusieurs machines.
- ✓ Tu lances un traitement → Hadoop envoie des petites tâches sur les nœuds qui détiennent les données.
- ✓ Il récupère les résultats et les assemble pour te donner le résultat final.

C'est rapide, robuste (même si des nœuds tombent) et évolutif (on peut ajouter des machines facilement).

30-Q : Quel est le lien entre Hadoop et MapReduce ?

R :

Hadoop est une plateforme qui permet de stocker et traiter de gros volumes de données de façon distribuée.

MapReduce est un **modèle de programmation** (et aussi un moteur d'exécution) qui est intégré dans Hadoop pour effectuer le traitement des

données.

En clair :

- Hadoop fournit le stockage (HDFS) et la gestion des ressources (YARN).
- MapReduce est un des moteurs que Hadoop utilise pour analyser les données stockées dans HDFS.

Donc MapReduce fonctionne *dans* Hadoop pour exécuter les tâches de calcul distribuées.

? 31-Q : Quels sont les avantages de Hadoop MapReduce ?

R :

- ✓ **Stockage massif** : il peut gérer des pétaoctets de données, grâce à HDFS qui distribue les données sur plusieurs machines.
- ✓ **Évolutif** : on peut facilement ajouter de nouvelles machines au cluster pour augmenter la capacité.
- ✓ **Tolérance aux pannes** : si une machine tombe en panne, Hadoop continue à fonctionner et récupère les données depuis un autre nœud.
- ✓ **Coût réduit** : il fonctionne sur des serveurs standards peu coûteux, pas besoin de matériel haut de gamme.
- ✓ **Traitement parallèle** : les données sont traitées en parallèle sur plusieurs nœuds, ce qui accélère les calculs.
- ✓ **Flexibilité** : il peut traiter toutes sortes de données (texte, images, vidéos, structurées ou non).

Si tu souhaites, je peux aussi donner des limites/inconvénients pour compléter.

? 32-Q : Quels sont les applications de Hadoop MapReduce ?

R :

-  **Analyse de logs** — analyser les fichiers journaux (web, serveurs, sécurité) pour trouver des tendances, erreurs, comportements d'utilisateurs.
-  **Indexation pour moteurs de recherche** — créer et maintenir les index de milliards de pages web, comme le faisait Google à ses débuts.
-  **Analyse de comportements d'achat** — étudier les transactions ou parcours clients dans le e-commerce pour recommandations ou promotions.

 **Bioinformatique** — analyser d'immenses volumes de données génétiques ou médicales pour recherche scientifique.

 **Analyse de réseaux sociaux** — extraire des tendances, hashtags, influenceurs à partir de flux massifs de tweets ou posts.

 **Traitement d'images/satellite** — analyser de grandes quantités d'images ou vidéos pour cartographie ou surveillance.

? 33-Q : Parle moi de l'écosystème Apache

R :

C'est l'ensemble des outils et projets autour de Hadoop, développés par la fondation Apache, pour faciliter le stockage, le traitement et l'analyse des Big Data.

Hadoop est la base (stockage et calcul), et tout un ensemble d'outils est venu s'ajouter pour couvrir des besoins spécifiques.

◆ Les plus connus dans l'écosystème Apache :

- **HDFS** — le système de fichiers distribué (la base pour stocker les données).
- **YARN** — pour gérer et allouer les ressources du cluster.
- **MapReduce** — pour calculer/traiter les données en parallèle.

Puis autour de ces 3 piliers, on a :

Pour le stockage et les bases :

- **HBase** — une base de données NoSQL (orientée colonnes) pour accéder rapidement à de gros volumes de données.
- **Hive** — pour écrire des requêtes SQL sur les données HDFS (les transforme en jobs MapReduce).
- **Sqoop** — pour importer/exporter des données entre bases relationnelles (MySQL, Oracle...) et Hadoop.
- **ZooKeeper** — pour la coordination et la gestion des configurations dans un cluster.

Pour l'ingestion et le traitement des flux :

- **Flume** — pour collecter des données en temps réel (par ex., logs) et les envoyer vers Hadoop/HBase.
- **Pig** — un langage de script (plus simple que Java) pour écrire des traitements de données.

Pour le calcul distribué plus moderne :

- **Spark** — moteur de traitement en mémoire, beaucoup plus rapide que MapReduce, avec des bibliothèques pour machine learning, graphes, SQL...
- **Storm** — pour le traitement en temps réel (flux de données).
- **Flink** — aussi pour le temps réel et le batch, très performant pour les deux.

En résumé :

- Hadoop (HDFS + YARN + MapReduce) est le socle.
- Autour de lui, des outils spécialisés pour :
 - Stocker (HBase, Hive)
 - Déplacer (Sqoop, Flume)
 - Coordonner (ZooKeeper)
 - Traiter plus facilement et rapidement (Pig, Spark, Storm, Flink...)

34-Q : Quels sont les principaux composants Hadoop ?

R :

 Les **principaux composants de Hadoop** sont au nombre de **3 (ou 4)**. Les voici clairement expliqués :

HDFS (Hadoop Distributed File System)

C'est le système de fichiers distribué de Hadoop.

Il découpe les fichiers en blocs, les distribue et les réplique sur plusieurs machines pour garantir fiabilité et disponibilité.

2 YARN (Yet Another Resource Negotiator)

C'est le gestionnaire de ressources de Hadoop.

Il alloue les ressources (CPU, mémoire) aux applications et planifie les tâches sur les nœuds du cluster.

3 MapReduce

C'est le modèle de calcul de Hadoop.

Il permet de traiter les données en parallèle en deux phases : *Map* (traitement local) et *Reduce* (agrégation des résultats).

(4) Parfois, on mentionne aussi :

Common (Hadoop Common)

Ce sont les bibliothèques et utilitaires partagés nécessaires pour faire fonctionner les 3 autres composants.



En résumé :

-  **HDFS** → Stocker les données
-  **YARN** → Gérer les ressources
-  **MapReduce** → Traiter les données
-  **Common** → Fournir les outils de base

Si tu veux, je peux aussi te faire un petit tableau ou schéma pour rendre encore plus visuel. Dis-le !

? 35-Q : Parle moi de l'architecture Hadoop

R :



Architecture de Hadoop

Hadoop est organisé en **3 couches principales**, qui travaillent ensemble dans un cluster pour stocker et traiter de grandes quantités de données.

1 Couche de stockage (HDFS)

- C'est la base.
 - Hadoop **Distributed File System** (HDFS) est un système de fichiers distribué.
 - Il divise les fichiers en **blocs**, les répartit sur plusieurs machines et les réplique pour assurer fiabilité et tolérance aux pannes.
 - Il permet de stocker des données volumineuses et d'y accéder rapidement.
-

2 Couche de gestion des ressources (YARN)

- **Yet Another Resource Negotiator (YARN)** gère l'utilisation des ressources du cluster.
 - Il sait combien de mémoire, CPU et autres ressources chaque application peut utiliser.
 - Il planifie et alloue ces ressources aux applications et aux tâches de calcul.
-

3 Couche de traitement (MapReduce)

- C'est la partie qui effectue le calcul sur les données stockées dans HDFS.
 - Il exécute des programmes en deux étapes (Map puis Reduce) pour analyser et transformer les données.
-



Résumé visuel :

Couche	Rôle principal
 HDFS	Stockage distribué et fiable
 YARN	Gestion des ressources et planification
 MapReduce	Traitement parallèle des données

Ces 3 couches sont déployées sur un **cluster Hadoop** (un ensemble de serveurs) pour fournir une plateforme robuste de Big Data.

? 36-Q : Qu'est-ce que HDFS et pourquoi existe-t-il ?

R :

HDFS (Hadoop Distributed File System) est le **système de fichiers distribué de Hadoop**.

Il a été conçu pour stocker des **fichiers très volumineux** (des gigas, téraoctets ou plus) sur un **cluster de serveurs ordinaires** (pas besoin de matériel haut de gamme).

Pourquoi ?

- Parce qu'un seul disque dur n'est ni assez grand ni assez fiable pour stocker des données massives.
- HDFS divise les fichiers en **blocs (64, 128 ou 256 Mo)** et les distribue sur plusieurs machines.
- Il réplique ces blocs (généralement 3 copies) pour éviter la perte en cas de panne d'une machine.

C'est donc une **solution fiable, scalable et tolérante aux pannes** pour le stockage de Big Data.

? 37-Q : Quels sont les composants principaux de HDFS et leur rôle ?

R :

HDFS suit une **architecture maître/esclave**, avec 3 principaux composants :

◆ **NameNode (maître)**

- Il gère les **métadonnées** : il sait quels fichiers existent, en quels blocs ils sont découpés, où se trouvent ces blocs.
- C'est comme un « carnet d'adresses » qui dit sur quelle machine est stocké chaque morceau de fichier.
- Il ne stocke pas les données elles-mêmes, juste les informations où les trouver.
- Il gère aussi les permissions et la réplication des blocs.

◆ **DataNodes (esclaves)**

- Ce sont les machines qui stockent **effectivement les blocs de données** sur leurs disques locaux.
- Ils reçoivent des instructions du NameNode pour créer, supprimer ou répliquer des blocs.
- Ils signalent régulièrement leur état au NameNode et envoient la liste des blocs qu'ils stockent.

◆ **Secondary NameNode**

- Il **n'est pas un secours en direct** du NameNode.
- Son rôle est d'aider le NameNode à éviter que ses journaux de modifications deviennent trop gros, en les fusionnant régulièrement avec l'état actuel des métadonnées.
- Il peut servir à reconstruire l'état du NameNode en cas de panne, mais avec un délai.

? 38-Q : Comment fonctionne HDFS ? (mécanisme général + accès aux données)

R :

◆ **Stockage :**

- Quand tu enregistres un fichier, il est découpé en blocs.
- Chaque bloc est envoyé à plusieurs DataNodes selon la règle de réplication (exemple : 3 copies, sur des machines/racks différents).
- Le NameNode garde une trace de quels blocs composent quel fichier et où sont placés les blocs.

◆ **Lecture d'un fichier :**

1 Le client demande au NameNode la liste des blocs et l'emplacement des DataNodes.

2 Le NameNode ne donne que les adresses, il ne transfère pas les données.

3 Le client se connecte directement aux DataNodes pour lire les blocs et reconstituer le fichier.

◆ **Écriture d'un fichier :**

- 1 Le client envoie la requête au NameNode, qui valide et réserve les blocs.
- 2 Le client envoie les blocs aux DataNodes, qui se chargent de les stocker et de les répliquer.
- 3 Une fois terminé, le NameNode met à jour ses métadonnées.

? 39-Q : Qu'est-ce qui rend HDFS fiable et efficace ?

R :

◆ **Répartition et parallélisme** : Les blocs sont répartis et traités en parallèle sur plusieurs machines.

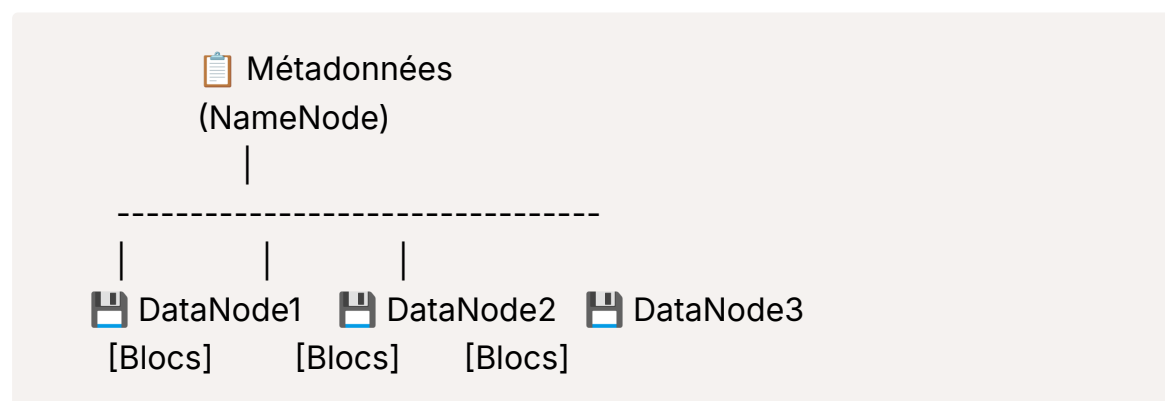
◆ **Réplication** : Chaque bloc est dupliqué pour éviter la perte en cas de panne.

◆ **Tolérance aux pannes** :

- Si un DataNode tombe, les blocs qu'il héberge sont recopiés sur d'autres nœuds.
- Si le NameNode tombe, le Secondary NameNode permet de restaurer une version récente.

◆ **Optimisation pour gros fichiers en lecture/écriture séquentielle** :

- Moins adapté pour des millions de petits fichiers ou des accès très fréquents en lecture aléatoire.



Tu as :

- Un chef d'orchestre (**NameNode**) qui connaît la carte du stockage.
- Des travailleurs (**DataNodes**) qui gardent les données et font le boulot.
- Un assistant (**Secondary NameNode**) qui nettoie les papiers du chef pour éviter qu'il ne sature.

? 40-Q : Qu'est-ce que MapReduce, et pourquoi en a-t-on besoin ?

R :

MapReduce est un modèle de programmation conçu par Google, et largement implémenté par Apache Hadoop, pour traiter de **très grandes quantités de données** de manière parallèle et distribuée sur un cluster de machines.

Tu en as besoin parce que :

- Les données sont trop volumineuses pour tenir sur une seule machine.
- Tu veux diviser le travail en petites tâches qui s'exécutent en parallèle.
- Tu souhaites que ton traitement soit **tolérant aux pannes**, et ne dépende pas d'une machine unique.

Il est basé sur deux fonctions clés :

- **Map** : transforme les données en paires (clé, valeur) et les trie.
- **Reduce** : regroupe les données par clé et résume ou agrège les résultats.



Représentation visuelle : imagine un chef d'orchestre (le JobTracker) qui divise la partition en plusieurs feuilles pour les musiciens (TaskTrackers), qui jouent chacun leur partie. À la fin, le chef récupère toutes les notes et en fait une symphonie cohérente.

? 41-Q : Comment fonctionne le mécanisme MapReduce étape par étape ?

R :

Un traitement MapReduce se déroule en plusieurs phases très claires :

◆ Étape 1 : Phase Map

- Les données d'entrée (souvent sur HDFS) sont découpées en blocs.
- Chaque bloc est lu par un **Map task (mappeur)**, qui produit des paires (clé, valeur).
- Ces paires sont triées et préparées pour être regroupées.

Exemple : compter les mots dans des textes → ("mot", 1)
pour chaque occurrence.

◆ Étape 2 : Phase de Shuffle & Sort (ou Phase de mélange)

- Toutes les paires issues des Map tasks sont rassemblées et triées par clé.
- Les mêmes clés sont regroupées pour être envoyées à la même tâche Reduce.

◆ Étape 3 : Phase Reduce

- Chaque **Reduce task (réducteur)** prend une clé et une liste des valeurs associées.
- Elle agrège ou résume les valeurs et produit le résultat final.



Représentation visuelle :

- Phase Map → couper en morceaux et trier par couleur.
- Shuffle → mettre ensemble tous les morceaux de la même couleur.
- Reduce → coller les morceaux de chaque couleur pour former un tableau complet.

? 42-Q : Quels sont les composants de l'architecture MapReduce ?

R :

On distingue deux types de composants : maîtres et esclaves, tous communiquant entre eux.

◆ Côté maître :

- **JobTracker (JT)** : supervise tout le travail, distribue les tâches aux nœuds, surveille les progrès, gère les ressources et relance les tâches échouées.
 - Il attribue les slots map ou reduce selon la disponibilité.

- Assure la **localité des données** autant que possible pour limiter le trafic réseau.

◆ Côté esclave :

- **TaskTracker (TT)** : s'exécute sur chaque nœud, reçoit les instructions du JobTracker, exécute les tâches (map/reduce) dans des slots dédiés.
 - Gère ses ressources locales (CPU/mémoire) et les rapporte au JT.
 - Peut exécuter plusieurs tâches en parallèle (selon le nombre de slots).



Représentation visuelle : le JobTracker est le chef de chantier qui attribue les briques aux maçons (TaskTrackers) et s'assure qu'ils travaillent bien.

? 43-Q : Quelles sont les forces et limites de MapReduce ?

R :

Forces :

- ✓ Parfait pour le **Big Data** (échelle en téra ou péta-octets).
- ✓ Massivement parallèle.
- ✓ Tolérant aux pannes (si un nœud échoue, le travail est relancé ailleurs).
- ✓ Exécution asynchrone et indépendante des tâches → facile à gérer.

Limites :

- ⊘ Pas optimal pour les petits jeux de données (overhead).
- ⊘ Nécessite d'écrire son propre algorithme pour chaque problème (pas d'interfaces « clé en main » comme SQL).
- ⊘ SPOF (Single Point of Failure) : le JobTracker peut devenir un goulot d'étranglement ou tomber en panne.

? 44-Q : Qu'est-ce que YARN et pourquoi est-il important ?

R :

YARN (Yet Another Resource Negotiator) est le **gestionnaire de ressources** et **planificateur de tâches** de l'écosystème Hadoop.

Il a été introduit pour remplacer l'ancien JobTracker/TaskTracker de Hadoop v1 et résoudre ses limitations :

- Il alloue et gère efficacement les ressources (CPU, RAM, disques, réseau) dans un cluster.
- Il permet d'exécuter plusieurs types d'applications sur Hadoop (pas seulement MapReduce) comme Spark, Hive, HBase, etc.
- Il sépare la **gestion des ressources** de la **gestion des tâches**, rendant le système plus flexible et évolutif.

? 45-Q : Quels sont les principaux composants de l'architecture YARN et leurs rôles ?

R :

◆ **Composant 1 : Resource Manager (RM)**

- S'exécute sur le nœud maître.
- Gère les ressources globales du cluster.
- Reçoit les demandes de ressources des applications et les distribue aux Node Managers.
- Se compose de :
 - **Planificateur (Scheduler)** : attribue les ressources selon des politiques (capacité, équité...).
 - **Application Manager** : gère la soumission des applications et le cycle de vie de leurs Application Masters.

◆ **Composant 2 : Node Manager (NM)**

- S'exécute sur chaque nœud esclave.
- Gère les ressources locales (CPU, RAM...) et exécute les conteneurs.
- Surveille la santé de son nœud et envoie des « heartbeats » (pulsations) réguliers au RM.
- Exécute et tue les conteneurs sur demande.

◆ **Composant 3 : Application Master (AM)**

- Processus spécifique à chaque application.
- Coordonne l'exécution de l'application dans le cluster.
- Demande des ressources au RM et lance les conteneurs via le NM.
- Suit l'état des tâches, gère les erreurs et notifie le RM une fois terminé.

◆ Composant 4 : Conteneur

- Unité d'exécution qui regroupe une quantité précise de ressources (mémoire, CPU, disque...).
- Géré et lancé par le NM.
- Permet à une tâche d'utiliser ces ressources sur un nœud donné.

? 46-Q : Comment se déroule le processus de soumission d'une application dans YARN ?

R :

Étapes principales :

- 1 Le client soumet une application au Resource Manager.
- 2 Le RM attribue un **Application ID** unique.
- 3 Un **contexte de soumission d'application** est créé.
- 4 Le RM réserve un conteneur pour lancer l'Application Master.
- 5 L'Application Master démarre et s'enregistre auprès du RM.
- 6 L'AM demande des conteneurs supplémentaires pour exécuter le code applicatif.
- 7 Le NM lance ces conteneurs et exécute le code.
- 8 Le client peut interroger le RM ou l'AM pour surveiller l'état de l'application.
- 9 L'AM se désenregistre auprès du RM lorsque l'application est terminée.

Composant	Rôle principal
Resource Manager	Gère et alloue les ressources globales.
Node Manager	Gère les ressources locales, exécute les conteneurs.
Application Master	Gère une application spécifique (cycle de vie, exécution, erreurs).
Conteneur	Regroupe les ressources nécessaires à une tâche.

? 50-Q : Comment fonctionne la planification dans YARN ?

R :

Le RM utilise un planificateur pour allouer les ressources aux applications :

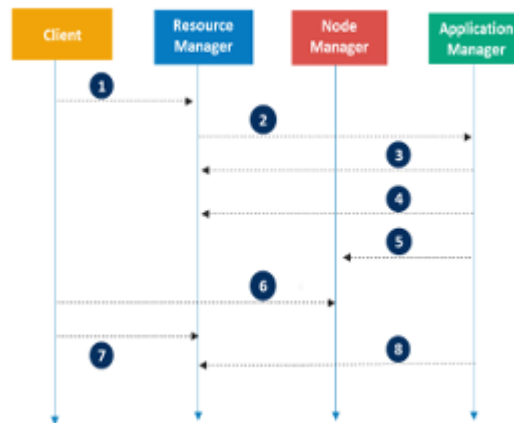
- **Capacity Scheduler** : assure qu'une certaine capacité minimale est réservée à chaque file.
- **Fair Scheduler** : essaie de partager équitablement les ressources entre toutes les applications.

? 51-Q : Quelles sont les étapes d'exécution typiques d'une application avec YARN ?

R :

Workflow simplifié :

- 1 Client → soumet l'application.
- 2 RM → réserve un conteneur et démarre l'Application Master.
- 3 AM → s'enregistre, demande des ressources supplémentaires.
- 4 RM → alloue des conteneurs à AM.
- 5 NM → lance les conteneurs avec le code applicatif.
- 6 AM → surveille les tâches, gère les échecs éventuels.
- 7 Application → se termine, AM se désenregistre.



? 52-Q : Qu'est-ce qu'Apache Spark et pourquoi est-il important ?

R :

Spark est un moteur open source de traitement de données big data, conçu à UC Berkeley, plus puissant et rapide que Hadoop/MapReduce classique.

- ✅ Il propose des calculs en mémoire (in-memory), ce qui accélère le traitement.
- ✅ Il prend en charge le batch, le streaming, le machine learning et les graphes.
- ✅ Il utilise des RDD (Resilient Distributed Dataset), des objets immuables et distribués.
- ✅ Idéal pour des algorithmes complexes d'exploration de données et d'apprentissage automatique.

? 53-Q : Quels sont les concepts clés de Spark ?

R :

- **RDD (Resilient Distributed Dataset)** : collection distribuée d'objets, immuable, tolérante aux pannes.
- **Variables de diffusion (Broadcast)** : envoient une seule copie d'une variable à tous les nœuds pour économiser de la bande passante.
- **Accumulateurs** : variables partagées pour l'agrégation, utilisées dans les actions.
- **Évaluation différée** : transformations planifiées, exécutées seulement quand une action est appelée.

? 54-Q : Quelle est l'architecture de Spark ?

R :

🏗️ 5 couches principales :

- 1 Stockage de données (HDFS, HBase...)
 - 2 Gestionnaire de ressources (YARN, Mesos...)
 - 3 Moteur principal Spark
 - 4 Bibliothèques : SQL, MLlib (machine learning), GraphX, Spark Streaming
 - 5 API : Java, Scala, Python, R
-

? 55-Q : Quels sont les avantages de Spark par rapport à Hadoop MapReduce ?

R :

- ✓ Plus rapide (in-memory, DAG optimisé)
 - ✓ Support du batch et du streaming
 - ✓ Bibliothèques riches pour ML, graphes, SQL
 - ✓ Plus flexible et plus simple à programmer
-

? 56-Q : Quels sont les défis ou limites de Spark ?

R :

- ⊗ Consomme beaucoup de mémoire
 - ⊗ Besoin de compétences spécialisées pour optimiser
 - ⊗ Certaines architectures ne sont pas encore optimales dans des environnements très hétérogènes
-

? 57-Q : Qu'est-ce qu'Apache Storm ?

R :

Storm est un framework open source pour le traitement **en temps réel (streaming)** des données.

- ✓ Idéal pour les flux continus de données (capteurs, logs, réseaux sociaux...).
 - ✓ Fonctionne en mode distribué sur un cluster.
 - ✓ Faible latence, haut débit.
-

? 58-Q : Quelle est l'architecture de Storm ?

R :

L'architecture d'**Apache Storm** repose sur une organisation distribuée composée de deux types principaux de nœuds, ainsi qu'un coordinateur central. Elle est conçue pour exécuter des **topologies** (flux de traitement en temps réel) sur plusieurs machines de façon fiable et scalable.

Les composants

◆ 1. Master Node (nœud maître)

- Il est responsable de la **gestion et de la supervision du cluster**.
 - Il surveille les défaillances, décide de la répartition des tâches et assigne les processus à exécuter sur les différentes machines.
 - Le logiciel qui s'exécute sur le master node s'appelle **Nimbus**.
 - Nimbus attribue les tâches aux nœuds ouvriers.
 - Il distribue le code des topologies et relance les tâches en cas d'échec.
-

◆ 2. Worker Node (nœud ouvrier)

- Ces nœuds exécutent effectivement les tâches de traitement assignées par le Nimbus.
 - Sur chaque worker node s'exécute un composant appelé **Superviseur (Supervisor)**.
 - Le superviseur reçoit les instructions de Nimbus et démarre les processus spécifiques pour traiter les données.
 - Chaque worker node peut exécuter plusieurs sous-processus d'une topologie.
-

◆ 3. Zookeeper

- Storm utilise **Apache Zookeeper** comme coordinateur entre le **Nimbus** (master) et les **Superviseurs** (workers).
- Zookeeper maintient l'état du cluster, assure la synchronisation et la communication entre les nœuds.
- Il garantit que les informations de configuration et les messages de contrôle sont correctement transmis et que la tolérance aux pannes est

assurée.

◆ Résumé

- **Nimbus** (sur le master) gère et distribue les tâches.
- **Superviseur** (sur les workers) exécute les tâches attribuées.
- **Zookeeper** coordonne la communication et la synchronisation entre Nimbus et les superviseurs.

? 59-Q : Quels sont les avantages et limites de Storm ?

R :

- ✓ Temps réel et rapide
- ✓ Hautement évolutif et tolérant aux pannes
- ✓ Facile à déployer et à surveiller
- ✗ Moins adapté au batch ou aux traitements complexes
- ✗ Pas optimisé pour les cas d'usage nécessitant des états complexes

? 60-Q : Qu'est-ce qu'Apache Flink ?

R :

Flink est un moteur open source pour le **traitement des données en flux et par lots**, avec une forte orientation temps réel et une gestion d'état avancée.

- ✓ Idéal pour des pipelines en continu (streaming) et pour des calculs batch.
- ✓ Supporte un très faible temps de latence avec une haute précision (exactly-once).

? 61-Q : Quelle est l'architecture de Flink ?

R :

🧱 Constituée de 5 couches :

- 1 Stockage : HDFS, fichiers locaux, etc.
- 2 Déploiement & gestion des ressources : planification, monitoring, allocation
- 3 Moteur central (noyau) pour le traitement temps réel
- 4 API : batch et streaming

5 Bibliothèques de haut niveau : Java, Scala

? 62-Q : Quels sont les avantages de Flink ?

R :

- ✓ Temps réel avec exactitude (exactly-once)
- ✓ Traite batch et streaming de manière unifiée
- ✓ Très performant pour les calculs avec état
- ✓ Tolérance aux pannes

? 63-Q : Quels sont les défis ou limites de Flink ?

R :

- ⊘ Plus complexe à apprendre que Spark
- ⊘ Plus récent → moins d'écosystème/maturité que Hadoop et Spark
- ⊘ Optimisé pour des cas de traitement continu

? 64-Q : Comparaison entre Spark, Storm et Flink

R :

Caractéristique	Spark	Storm	Flink
Mode de traitement	Batch + Streaming	Streaming (temps réel)	Streaming + Batch
Données en mémoire	Oui (RDD, DAG)	Non	Oui
Cas d'usage typiques	ML, graphes, SQL, ETL	Monitoring, alertes	Pipelines continus, états
Tolérance aux pannes	Oui	Oui	Oui (avec exactly-once)
Facilité d'apprentissage	Moyenne	Simple	Complexe
Latence	Basse (mais + haute que Storm)	Très basse	Très basse

📌 En résumé :

- Utilise **Spark** si tu as besoin d'analytique avancée, ML, SQL, et traitements rapides.

- Utilise **Storm** pour des flux de données simples, à très faible latence.
- Utilise **Flink** si tu veux un moteur robuste pour du streaming avancé avec état et batch unifié.

? 65-Q : C'est quoi Hive ?

R :

- Hive est un **outil open-source**, initialement développé par **Facebook**, puis rendu public sous Apache.
- C'est une **surcouche SQL pour Hadoop**, qui permet d'écrire des requêtes de type SQL (appelé **HiveQL**) sur des données stockées dans des clusters Hadoop.
- Conçu pour les **analystes métier** et les **développeurs SQL**, il facilite :
 - Les **requêtes ad hoc**.
 - La **synthèse et l'analyse** de grands ensembles de données.
 - Le traitement de données stockées dans **HDFS** (ou autres systèmes compatibles Hadoop).
- Il agit comme un **entrepôt de données (Data Warehouse)** pour Hadoop et permet donc de faire de l'analyse de données de manière déclarative.

? 66-Q : C'est quoi Pig ?

R :

- Pig est aussi un **outil open-source**, cette fois créé par **Yahoo**, et également publié sous Apache.
- Contrairement à Hive, Pig propose une **langue de script** (appelée **Pig Latin**) pour le traitement de données.
- Conçu pour les **développeurs** et les **data engineers**, il est plus flexible que Hive et permet :
 - De décrire des flux de données complexes.
 - D'écrire des **programmes procéduraux** pour transformer, analyser et traiter le Big Data.
 - D'intégrer des **fonctions personnalisées** pour des besoins spécifiques.

- C'est donc une **plateforme de programmation haut niveau** pour traiter les données Hadoop, plus adaptée aux développeurs qu'aux analystes SQL.

? 67-Q : Comment se fait la communication des composants HDFS et MapReduce (MR) ?

R :

Dans un cluster Hadoop classique (v1), il y a des **composants maîtres** et des **composants esclaves** qui communiquent pour assurer le stockage et le traitement des données.

◆ **Composants maîtres :**

- **NameNode (NN)** : maître de HDFS, il gère la **métadonnée** (emplacement des blocs, permissions...).
- **Secondary NameNode (SNN)** : aide le NN à sauvegarder et fusionner ses métadonnées (il n'est pas un backup direct du NN !).
- **JobTracker (JT)** : maître de MapReduce, il gère la soumission et la planification des jobs, et surveille les tâches.

◆ **Composants esclaves :**

- **DataNode (DN)** : stocke les blocs de données physiques sur disque et répond aux commandes du NN.
- **TaskTracker (TT)** : exécute les tâches Map et Reduce planifiées par le JT et utilise les blocs fournis par le DN.

◆ **Communication entre ces composants :**

✓ **Entre maîtres :**

- Le **NameNode (NN)** et le **Secondary NameNode (SNN)** échangent pour la gestion des journaux d'édition et des checkpoints.
- Le **JobTracker (JT)** ne discute pas directement avec le NN ou le SNN mais peut dépendre des informations sur l'état des blocs via le TT.

✓ **Maîtres → esclaves :**

- Le **NN** envoie des instructions aux **DN** pour lire/écrire des blocs et reçoit des rapports réguliers sur la santé et la disponibilité des blocs.

- Le **JT** planifie les tâches sur les **TT** et reçoit des rapports sur la progression des jobs.

✅ **Esclaves → esclaves :**

- Le **TaskTracker (TT)** parle aux **DataNode (DN)** pour récupérer les blocs de données nécessaires au traitement.

? **68-Q : C'est quoi le NoSQL ?**

R :

NoSQL signifie *Not Only SQL*.

C'est un système de gestion de bases de données **non relationnelles**, conçu pour être :

- plus flexible (schéma souple ou inexistant),
- capable de gérer de grandes quantités de données (Big Data),
- performant sur des données en temps réel ou distribuées (web logs, IoT...),
- et évolutif horizontalement (ajout facile de serveurs).

NoSQL peut stocker des données :

- structurées,
- semi-structurées (XML, JSON...),
- ou non structurées (texte, images...).

? **69-Q : Quels sont les types de bases NoSQL ?**

R :

Il existe quatre grands types de bases de données NoSQL, chacune adaptée à un besoin particulier 📌

1 Bases orientées colonnes

- Elles stockent les données **par colonnes** plutôt que par lignes.
- Idéal pour les analyses en masse (Big Data, indexation, rapports...).

📌 **Exemples populaires :**

- **HBase**

- **Cassandra**
- **Hypertable**
- **Accumulo**

 Meilleures applications :

- **HBase** : accès en temps réel, opérations en bloc.
 - **Cassandra** : forte charge en écriture.
 - **Accumulo** : contrôle d'accès précis au niveau de la cellule.
-

2 Bases orientées documents

- Elles stockent les données sous forme de **documents JSON** (ou similaires).
- Les documents sont auto-descriptifs, ce qui rend ces bases très flexibles.

 Exemples populaires :

- **MongoDB**
- **CouchDB**
- **Terrastore**
- **RavenDB**

 Meilleures applications :

- **MongoDB** : requêtes dynamiques, indexation, grosses bases.
 - **CouchDB** : données peu changeantes, accumulation, cas web/mobile.
-

3 Bases orientées graphes

- Elles utilisent la **théorie des graphes**, avec des **nœuds** et des **arêtes**, pour représenter des relations complexes.

 Exemples populaires :

- **Neo4j**
- **InfiniteGraph**
- **InfoGrid**

- **HypergraphDB**
- **AllegroGraph**

🎯 Meilleures applications :

- Réseaux sociaux, moteurs de recommandation, détection de fraudes, où les relations sont importantes.

4 Bases clé-valeur

- Elles stockent les données sous forme de paires **clé → valeur**.
- Très simples et très rapides pour des lectures/écritures massives.

📌 Exemples populaires :

- **Redis**
- **Riak**
- **Voldemort**
- **Infinispan**
- **Hazelcast**

🎯 Meilleures applications :

- **Redis** : données très dynamiques, statistiques, cache.
- **Riak** : haute disponibilité, tolérance aux pannes.
- **Voldemort** : gros volumes de données.

◆ Type	📁 Exemples	🎯 Idéal pour...
Orienté colonne	HBase, Cassandra, Accumulo	Big Data, accès rapide en lecture/écriture, analytique
Orienté documents	MongoDB, CouchDB	Applications web, données JSON, flexibilité
Orienté graphes	Neo4j, AllegroGraph	Relations complexes, réseaux sociaux
Clé-valeur	Redis, Riak, Voldemort	Données rapides, cache, sessions

? 70-Q : Approche classique vs Approche Big Data

R :

Aspect	Approche classique	Approche Big Data
 Type de données	Données structurées (tables, colonnes, lignes).	Données non structurées, semi-structurées et structurées.
 Schéma	Schéma rigide, défini à l'avance.	Sans schéma ou schéma flexible.
 Base de données	Relationnelle (SQL, transactions, contraintes).	NoSQL : orientée colonnes, documents, clé-valeur, graphes.
 Flux de données	Données statiques , enregistrées et analysées après coup.	Données en continu (streaming) , analysées en temps réel.
 Relations	Optimisée pour gérer des relations complexes entre données.	Optimisée pour gérer de grandes quantités de données , pas nécessairement relationnelles.
 Évolution	Migration ou adaptation plus coûteuse et lente.	Évolution rapide grâce à des schémas flexibles.
 Analyse	Fonctionne bien pour des volumes modérés de données.	Fonctionne mieux pour des volumes massifs et variés.
 Efficacité	Moins efficace avec des volumes très grands ou des flux rapides.	Conçue pour être scalable et rapide avec des volumes énormes.

? 71-Q : Étude de cas de solutions Big Data

R :

Voici comment on met en place une **infrastructure Big Data** typique 📌

◆ Étapes et composants :

1 Sources de données :

- Plusieurs sources variées : logs web, IoT, réseaux sociaux, transactions financières...
- Ces sources génèrent des données hétérogènes.

2 Extraction - Transformation - Chargement (ETL) :

- Une **couche d'intégration** collecte, nettoie, transforme et charge les données.
- Assure la qualité et la cohérence avant analyse.

3 Stockage :

- Données stockées dans des :
 - bases relationnelles (Data marts, entrepôts de données)
 - bases NoSQL
 - systèmes distribués (HDFS...).

4 Traitement et analyse :

- Moteurs de traitement Big Data (Hadoop, Spark, Flink...).
- Les données sont traitées (batch ou streaming) et analysées selon les besoins.

5 Visualisation et reporting :

- Résultats transformés en rapports, dashboards et visualisations pour la prise de décision.

6 Infrastructure matérielle :

- Infrastructure physique robuste et redondante pour assurer haute disponibilité.
- Gestion des pannes et tolérance aux erreurs intégrées.



Figure : Infrastructure Big Data

Exemple typique :

- Une grande enseigne e-commerce collecte en continu les comportements clients (clics, achats, abandons).
 - Données traitées en streaming pour personnaliser les recommandations en temps réel.
 - Données historisées pour analyser les tendances de consommation sur plusieurs années.
 - Résultats présentés via des dashboards pour les équipes marketing et logistique.
-